Lab 1: The Morse Code Lock

GPIO and the Memory Map

Overview

The development board becomes a combination lock. The key is your first name in Morse code. You enter your name on the user button B1 as a sequence of short and long presses. The program decodes the presses and compares the result with the stored name. The user LED LD2 shows the result (Figure 1):

- If the name is correct, LD2 blinks quickly. The lock is open.
- If the name is not correct, LD2 stays on. The lock stays closed.

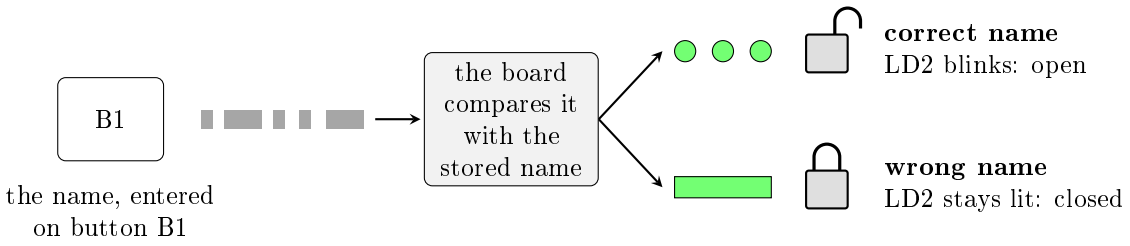


Figure 1: The Morse code lock. The grey bars show short and long presses of button B1. The program compares the entered name with the stored name and shows the result on LD2.

Structure of this laboratory

This laboratory has three parts.

- **Part "A", inline assembly.** Make LD2 blink with assembly code. With the oscilloscope, measure the time that LD2 stays on.
- **Part "B", register access in the C language.** Control the blink rate with a push-button. Then measure the contact bounce of the input, and learn how to remove it.
- **Part "C", the hardware abstraction layer (HAL).** Build the lock with the HAL functions from STMicroelectronics (ST).

Objectives

- To show that memory-mapped registers control a general-purpose I/O (GPIO) pin, and that load and store instructions access these registers.
- To find the address, the bit fields, and the reset value of a register in the reference manual.
- To configure and drive a GPIO pin with inline assembly.
- To predict a software delay from instruction cycle counts, and to measure the delay with an oscilloscope.
- To measure how much time a value in SRAM costs, compared with a value in a core register.
- To read a push-button through its registers, and to debounce it.
- To measure the duration of button presses with the HAL, and to decode Morse code.

Required Hardware

- The NUCLEO-G491RE development board.
- An oscilloscope.
- Three male-to-male jumper wires.

Pre-Laboratory Exercises

Do the pre-laboratory exercises before the laboratory session. The exercises of each part are after its background: Part "A", Part "B", and Part "C".

Project setup

Create a new project, as Step 2 of Lab 0 shows. Do not copy your code of Lab 0 into the new project. The generated code sets the core clock to **170 MHz**. Use this frequency in each timing calculation.

Put your code in `Core/Src/main.c`, in the section that each task gives. The file has two `while (1)` loops. Use the loop of `main()`. The loop of `Error_Handler()` runs only after an error. In the section names, `PV` means private variables, and `PD` means private defines.

Part "A": Inline Assembly

Background for Part "A"

To control an LED, a program writes to a memory address. The user LED LD2 is connected to pin PA5. Bit 5 of `GPIOA_ODR`, the output data register of port *A*, sets the output level of PA5. The address of this register is `0x4800 0014` (Figure 2). The prefix `0x` marks a hexadecimal number. Some assemblers write the same number with the suffix `h`: `48000014h`. When the clock of port *A* is on and PA5 is an output, a `1` in bit 5 turns LD2 on. The value `0` turns LD2 off. In Part "A", you do each of these operations with assembly instructions.

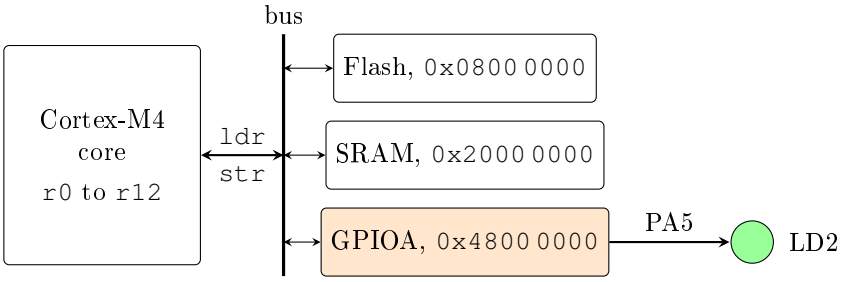


Figure 2: The Cortex-M4 core uses the same bus and the same `ldr` and `str` instructions to access flash, SRAM, and the registers of GPIO port *A*.

General-purpose I/O

The configuration of a microcontroller pin sets its function. The pin can be a *digital* output: the program sets its level to one of two values, high or low. PA5, which drives LD2, is an example. The pin can also be a digital input, or a connection to an on-chip peripheral. Thus, the pins are "general-purpose" (RM0440 section 9.2, p. 345). The pins are in groups, the *ports*. Each port has a letter, from *A* to *G*, and up to sixteen pins: PA5 is pin 5 of port *A*. Each port has its own base address and its own set of registers (RM0440 section 9.1, p. 345).

Memory-mapped I/O

The chip has hardware blocks outside the processor core, the *peripherals*: a GPIO port is an example. The program controls a peripheral through its *peripheral registers*. Each of these registers has a memory address, in the range `0x4000 0000` to `0x5FFF FFFF` (RM0440 section 2.2.2, p. 83). For example, a write to a GPIO register changes the state of a pin. The level of the pin comes from a read of the register. Because the registers have addresses, the processor accesses them as memory. The *load* instruction reads a register, and the *store* instruction changes it.

Core registers and memory

The processor has two types of storage (Figure 2):

- *core registers*, `r0` to `r12`: thirteen 32-bit registers in the processor, with no address.
- *memory*, which the processor accesses over the bus at an address: flash from `0x0800 0000`, SRAM from `0x2000 0000`, and the memory-mapped peripherals from `0x4000 0000` (RM0440 section 2.2.2, p. 83).

The global variables of a C program are in SRAM: the linker gives each variable an address there.

The Cortex-M4 is a *load/store* machine. Arithmetic and logic instructions, for example `subs`, `orr`, `bic`, and `eor`, operate only on core registers. Only `ldr` and `str` access memory.

Thus, a change to a value in memory usually has three steps. The processor loads the value into a core register with `ldr`, changes it there, and stores it with `str`. This is true for a counter in SRAM and for a GPIO register.

The registers that this lab uses

The address of a register of port A is the base address of port A, `0x4800 0000`, plus an offset from the manual. Part "A" uses three registers (Table 1):

- **`RCC_AHB2ENR`, the clock gate.** This register is in the reset and clock control block (RCC). It enables the peripheral clocks on AHB2, one of the three Advanced High-performance Buses (AHB) of the chip (RM0440 section 7.4.15, p. 306). The reset value is `0x0000 0000`, thus all ports are disabled after a reset. Bit 0, GPIOAEN, enables port A.
- **`GPIOA_MODER`, the mode register.** It has two bits for each pin, which set the mode of the pin (RM0440 section 9.4.1, p. 355).
- **`GPIOA_ODR`, the output data register.** It has one bit for each pin. The bit holds the level that the pin drives as an output (RM0440 section 9.4.6, p. 357).

Register	Address	Purpose
`RCC_AHB2ENR`	`0x4002 104C`	Clock gate. Bit 0 (GPIOAEN) enables port A.
`GPIOA_MODER`	`0x4800 0000`	Pin mode, 2 bits for each pin.
`GPIOA_ODR`	`0x4800 0014`	Output level, 1 bit for each pin.

Table 1: The three registers that Part "A" uses. Each address is the base address of port A plus the offset from the manual. `RCC_AHB2ENR` is different: it is in the RCC block at `0x4002 1000`.

Fact 1: The G4 memory map is different from the F4 memory map

On the STM32F4, the GPIO clocks are on `AHB1`, and `GPIOA` starts at `0x4002 0000`. On the STM32G4, the GPIO clocks are on `AHB2`, and `GPIOA` starts at `0x4800 0000`. Most example code on the internet is for the F4. F4 addresses compile on a G4 and cause **no error**. The writes go to the wrong peripheral or to an unused address, and the pin does not change. Compare each address with Table 1 or with the reference manual.

Fact 2: A peripheral ignores writes until its clock is enabled

If you write to `GPIOA_MODER` before GPIOAEN is 1, no fault and no warning occur. The only symptom is a pin that does not change. In this project, the generated `MX_GPIO_Init()` runs before your code. It enables the GPIOA, GPIOB, GPIOC, and GPIOF clocks. Thus, if you remove your clock enable, you see no change.

The mode of a pin

In a register, a *field* is a group of adjacent bits that holds one setting. Each pin n has a 2-bit field in `MODER`, at bits $2n+1$ to $2n$. PA5 (LD2) uses bits 11 to 10 (Figure 3).

15 [31:30]	14 [29:28]	13 [27:26]	12 [25:24]	11 [23:22]	10 [21:20]	9 [19:18]	8 [17:16]	7 [15:14]	6 [13:12]	5 [11:10]	4 [9:8]	3 [7:6]	2 [5:4]	1 [3:2]	0 [1:0]

Figure 3: The fields of `GPIOA_MODER`. The top row gives the pin number, and the bottom row gives the bit range. The field of pin 5 (bits 11 to 10) is orange.

The register description (RM0440 section 9.4.1, p. 355) gives four values for each field:

- **`01`, general-purpose output** (RM0440 section 9.3.10, p. 351): the output driver drives the level in `ODR`. The default output type is push-pull. For PA5:

```
"bic   r5, r5, #0xC00    \n"   /* clear bits 11:10 */
"orr   r5, r5, #0x400    \n"   /* set bits 11:10 to 01 */
```

- **`00`, input** (RM0440 section 9.3.9, p. 351): the level of the pin goes through a Schmitt trigger into the input data register, `GPIOx_IDR` (RM0440 section 9.4.5, p. 357). The value `00` has only zeros, thus it is enough to clear the field:

```
"bic   r5, r5, #0xC00    \n"   /* clear bits 11:10 to 00: input */
```

- **`10`, alternate function** (RM0440 section 9.3.11, p. 352): an on-chip peripheral drives the pin. `GPIOx_AFRL` and `GPIOx_AFRH` select the peripheral. The generated code uses this mode on PA2 for the virtual COM port, the serial link to your computer through the ST-LINK:

```
"bic   r5, r5, #0x30     \n"   /* clear bits 5:4 (PA2's field) */
"orr   r5, r5, #0x20     \n"   /* set bits 5:4 to 10 */
```

- **`11`, analog** (RM0440 section 9.3.12, p. 353): the output driver and the Schmitt trigger of the input are off. Thus, the pin uses no power at any voltage on it. This mode is the reset value of most pins, for example PA5. Fact 3 gives the exceptions on port *A*.

Read-modify-write

The field of a pin shares its 32-bit register with the fields of fifteen other pins. An `str` instruction writes all 32 bits. Thus, to change one field, do the three steps in Core registers and memory:

1. Load the full register into a core register with `ldr`.
2. Clear the field with `bic`. Then set it with `orr`. The other bits do not change.
3. Store the result with `str`.

This sequence is called *read-modify-write*. The clear is necessary, because `orr` can only set bits. Thus, `orr` cannot change `11` or `10` into `01`. PA5 resets to `11` (analog). Without the `bic`, PA5 stays in analog mode.

Fact 3: One register holds the settings of sixteen pins

`GPIOA_MODER` also holds the fields of these pins:

- PA13, PA14, and PA15, the debug pins. PA13 and PA14 carry Serial Wire Debug (SWD). SWD is the two-wire link that the ST-LINK uses to program and debug the board. The three pins are in alternate function mode after a reset (RM0440 section 9.3.1, p. 348).
- PA2 and PA3, which carry the virtual COM port.

A plain `str` of `0x400` to `GPIOA_MODER` configures PA5 correctly. It also sets all the other fields to `00`. The debug pins leave alternate function mode, and **the debugger disconnects**. Always read, modify, and write back.

The same rule applies to `ODR`.

Inline assembly

In Part "A", you write assembly in `main()` with the `__asm volatile` statement of the GNU C compiler. The compiler gives the instructions to the assembler without changes. Thus, the processor executes exactly the instructions that you write.

- Each instruction is a C string that ends in `\n`. The compiler joins adjacent strings into one block, with one instruction on each line.
- The keyword `volatile` informs the compiler that it must not move or remove the block. Without `volatile`, the compiler can remove the block, because no C code uses a result of it.
- Three colons separate three lists: the output operands, the input operands, and the *clobber list*. The clobber list names each core register that the block changes, and also `"cc"` (the condition flags) and `"memory"`. The compiler then keeps no value of its own in these registers. Part "A" leaves the first two lists empty. If the clobber list is not complete, the code works only by chance.
- Labels are numbers. `1:` defines a label. `1b` refers to the nearest label 1 before it, and `1f` refers to the nearest label 1 after it. Listing 1 has the labels 1 and 2, with the branches `bne 2b` and `b 1b`. You can use a numeric label again. You cannot use a named label again, because the compiler can duplicate the block.
- `ldr rd, =value` loads a 32-bit constant or an address. A 32-bit value does not fit in one instruction. Thus, the assembler stores the value in flash, in a *literal pool*, and the instruction loads it from there. The `.ltorg` line puts the literal pool at the end of the block.

Instruction	Effect
`ldr rd, =value`	Load a 32-bit constant or an address into `rd`.
`ldr rd, [rn, #off]`	Load the word at address `rn + off` into `rd`.
`str rd, [rn, #off]`	Store `rd` at address `rn + off`.
`orr`, `bic`, `eor`	Bitwise OR, bit clear (AND NOT), and exclusive OR.
`subs rd, rn, #n`	Subtract `n` from `rn`, and set the condition flags.
`b`, `bne`	Branch always, and branch if the result is not zero.

Table 2: The Cortex-M4 instructions that Part "A" uses.

Pre-Laboratory Exercises for Part "A"

1. **Find the registers in the reference manual.** In RM0440, find the section about the AHB2 peripheral clock enable register. Then find the section about the GPIO port mode register. Use these two sections to give:
   (a) the bit number of GPIOAEN.
   (b) the address offset of each of the two registers.
   (c) the bit range of the PA5 field in `MODER`.
   (d) the two-bit value that selects general-purpose output.

2. **Predict the timing.** Tasks A1 and A2 measure the *half-period* of the blink. The half-period is the time that LD2 stays on. The two tasks do the same delay loop $N = 56\,666\,667$ times on a 170 MHz core. Only the instructions in one pass are different:

Task	One pass of the loop
A1, count in a register	`subs`, `bne`
A2, count in SRAM	`ldr`, `subs`, `str`, `bne`

   (a) Write an equation for the half-period. Use N, the number of clock cycles per pass, c, and the clock frequency, f.
   (b) Find the cycle count of each instruction in the Arm Cortex-M4 Technical Reference Manual. Use Table 3-1 (DDI 0439 section 3.3.1, p. 29) and the section *Load/store timings* (DDI 0439 section 3.3.2, p. 36), which changes some values of the table. For `bne`, use the value for a taken branch.
   (c) Calculate c for each task. Then predict the two half-periods and their ratio. If the manual gives a range, give your prediction as a range.
   (d) Explain why the loop must use `subs` and not `sub` (PM0214).

Task A1: Blink LD2 with assembly code

Run the code below without changes. It makes LD2 blink. Measure the duration of each half-period with the oscilloscope.

```
__asm volatile (
    /* RCC_AHB2ENR: set GPIOAEN (bit 0) */
    "ldr   r6, =0x4002104C   \n"
    "ldr   r5, [r6]          \n"
    "orr   r5, r5, #0x1      \n"
    "str   r5, [r6]          \n"

    /* GPIOA_MODER: MODE5 (bits 11:10) = 01, clear then set */
    "ldr   r0, =0x48000000   \n"   /* r0 = GPIOA base */
    "ldr   r5, [r0]          \n"
    "bic   r5, r5, #0xC00    \n"
    "orr   r5, r5, #0x400    \n"
    "str   r5, [r0]          \n"

    /* GPIOA_ODR (base + 0x14): set bit 5, LD2 on */
    "ldr   r5, [r0, #0x14]   \n"
    "orr   r5, r5, #0x20     \n"
    "str   r5, [r0, #0x14]   \n"

    /* One half-period, then flip bit 5 and do it again */
    "1:                      \n"
    "ldr   r3, =56666667     \n"   /* passes per half-period */
    "2:                      \n"
    "subs  r3, r3, #1        \n"
    "bne   2b                \n"
    "ldr   r5, [r0, #0x14]   \n"
    "eor   r5, r5, #0x20     \n"
    "str   r5, [r0, #0x14]   \n"
    "b     1b                \n"
    ".ltorg                  \n"   /* literal pool, never executed */
    :
    :
    : "r0", "r3", "r5", "r6", "cc", "memory"
);
```

Listing 1: Task A1: The code enables the GPIOA clock, configures PA5 as an output, and makes LD2 blink, in inline assembly. Put the block in the while (1) loop of main().

Structure of the code. Blank lines divide the code into four groups. The groups do these operations:

- enable the GPIOA clock in `RCC_AHB2ENR`.
- configure PA5 as an output in `GPIOA_MODER`.
- turn LD2 on in `GPIOA_ODR`.
- blink: count `r3` down from 56 666 667, toggle PA5, and do it again.

Each of the first three groups is a read-modify-write sequence.

Steps.

1. Put the code in `main.c`, between `USER CODE BEGIN WHILE` and `USER CODE END WHILE`. Build the project. Flash the board.
2. Make sure that LD2 comes on and then blinks.
3. Probe PA5. Measure the half-period. Compare it with your prediction from Pre-Laboratory Exercise 2(c).
4. Use Table 2 to find the effect of each instruction in the code.

Task A2: Move the count into SRAM and measure the cost

Store the count in memory, and measure the increase of the half-period. The count stays 56 666 667. Thus, the only difference from Task A1 is where the count is: in SRAM, and not in a core register.

```
uint32_t blink_count;   /* USER CODE BEGIN PV; lives in SRAM */

        "ldr   r4, =blink_count  \n"   /* r4 = its address */
```

Listing 2: Task A2: The global variable for the count, in SRAM, and the instruction that loads its address.

Steps.

1. Declare `blink_count` in the `USER CODE BEGIN PV` section, as the code above shows.
2. Before the blink loop, load the address of `blink_count` into `r4` one time. Add `r4` to the clobber list.
3. Change the delay loop so that `blink_count` holds the count. At the start of each half-period, store the count in `blink_count`. On each pass, load the count, subtract 1, and store it again. **Do not change the other code.**
4. Build the project. Flash the board. Measure the half-period with the oscilloscope, as in Task A1. Calculate its ratio to the half-period of Task A1.
5. In the debugger, find the address of `blink_count`. Find the region of the memory map that contains this address.
6. For each of the two loops, calculate the number of cycles per pass. Use the count, the clock frequency of 170 MHz, and the half-period that you measured.
7. Compare the two measurements with your predictions from Pre-Laboratory Exercise 2. Explain the difference between the two loops. Use the instructions in each pass, and where each instruction gets its data: a core register or memory.

Part "B": Register Access in the C Language

Background for Part "B"

To get the state of a button, a program reads from a memory address. The user button B1 is connected to pin PC13. Bit 13 of `GPIOC_IDR`, the input data register of port *C*, shows the level of this pin. The address of `GPIOC_IDR` is `0x4800 0810`. The bit is `1` when button B1 is pressed, and `0` when it is released.

In Part "B", you write C code that accesses each register through its address (Register access in the C language). Do not use `HAL_*` functions. The program operates as follows:

- At reset, LD2 is off.
- The first press starts a slow blink.
- Each press after the first press divides the half-period by 2. There are five speeds. At the fastest speed, the next press sets the blink to the slowest speed again.
- The program counts loop passes to time the blink, as in Part "A". The loop does not wait: it reads the input on each pass.

Task B1 uses the user button as the input. Tasks B2 and B3 use two jumper wires. One of the wires is on PA10.

Register access in the C language

In the C language, a macro changes the address of a register into an expression. The program reads and writes this expression as a variable:

```
#define GPIOC_IDR  (*(volatile uint32_t *)0x48000810UL)
```

- On the right side of an assignment, `GPIOC_IDR` reads the register. On the left side, it writes the register. Each access compiles to an `ldr` or an `str` (Table 2). Thus, `GPIOA_MODER |= ...` is still a read, a modification, and a write.
- `volatile` makes the compiler do each access in the source, in the same sequence. Without `volatile`, the compiler can read an input only one time and use the value again. It can also remove a write if the program does not read the result.

The registers of Table 3 read inputs, enable a pull-down, and set outputs with one write:

- `GPIOx_BSRR` sets or resets individual pins with one write (RM0440 section 9.4.7, p. 357).
- `GPIOx_IDR` holds the level of each input pin (RM0440 section 9.4.5, p. 357).
- `GPIOx_PUPDR` connects an internal pull-up or pull-down resistor to each pin. It has two bits for each pin (RM0440 section 9.4.4, p. 356).

Register	Address	Purpose
`GPIOA_PUPDR`	`0x4800 000C`	Pull-up or pull-down of each port *A* pin.
`GPIOA_IDR`	`0x4800 0010`	Input level of each port *A* pin.
`GPIOA_BSRR`	`0x4800 0018`	Sets or resets individual port *A* pins with one write.
`GPIOC_MODER`	`0x4800 0800`	Mode of each port *C* pin.
`GPIOC_IDR`	`0x4800 0810`	Input level of each port *C* pin.

Table 3: The registers for inputs and for output with one write. Port *C* starts at `0x4800 0800`. `RCC_AHB2ENR` enables its clock, as for port *A*.

Push-pull outputs

Two switches drive an output pin. One switch connects the pin to ground. The other switch connects the pin to V_{DD}, the supply voltage of the chip (3.3 V on this board). Each switch is a transistor. At all times, one switch is closed and the other switch is open (RM0440 section 9.3.10, p. 351). An `ODR` bit of `0` closes the switch to ground, and the pin is at 0 V. The value `1` closes the switch to V_{DD}, and the pin is at V_{DD} (Figure 4).

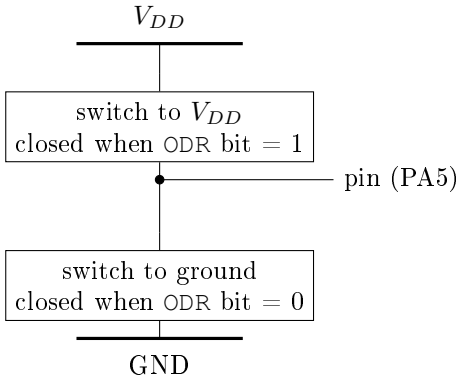


Figure 4: Push-pull output.

Because the pin is always driven, it does not need an external pull resistor. The other output type, open-drain, has only the switch to ground. It needs an external pull-up resistor for a high level. `GPIOx_OTYPER` selects the output type. Its reset value selects push-pull for all pins, thus you do not write it.

The Schmitt trigger input

The input circuit compares the voltage of the pin with a threshold voltage. A voltage above the threshold reads `1`, and a voltage below it reads `0`. With one threshold, a slow or noisy signal near the threshold gives a burst of `0`s and `1`s. To prevent this, each STM32G4 GPIO input goes through a *Schmitt trigger*, which has two threshold voltages (RM0440 section 9.3.9, p. 351, Figure 5):

- After a `1`, the input changes to `0` only when the voltage decreases below the lower threshold, V_{IL}.
- After a `0`, the input changes to `1` only when the voltage increases above the higher threshold, V_{IH}.

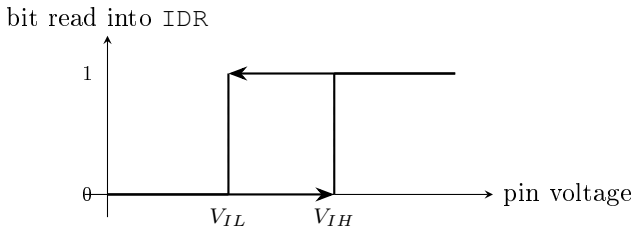


Figure 5: Schmitt trigger hysteresis. A signal between the two thresholds cannot make the bit in `IDR` oscillate.

The input from button B1

The user button B1 is connected to PC13 (UM2505 section 7.6.2, p. 23), as Figure 6 shows. Button B1 is **active high**: a press gives a high voltage, which the pin reads as `1`. When button B1 is released, no part drives PC13. An external *pull-down* resistor, which connects the pin to ground, then holds PC13 at 0 V, and the pin reads `0`. When button B1 is pressed, it connects PC13 to V_{DD}, and the pin reads `1`. The level goes through the Schmitt trigger into bit 13 of `GPIOC_IDR`.

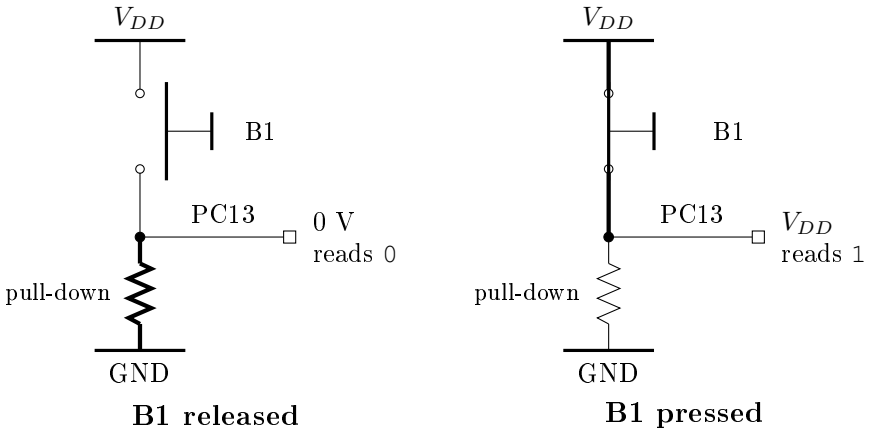


Figure 6: How button B1 sets the level of PC13 (simplified). The thick line shows the path that sets the level of the pin.

There are two different quantities (Figure 7):

- the *level*: the state of button B1 at this time, as bit 13 shows it.
- a *press*, or *rising edge*: the time at which the level changes from `0` to `1`.

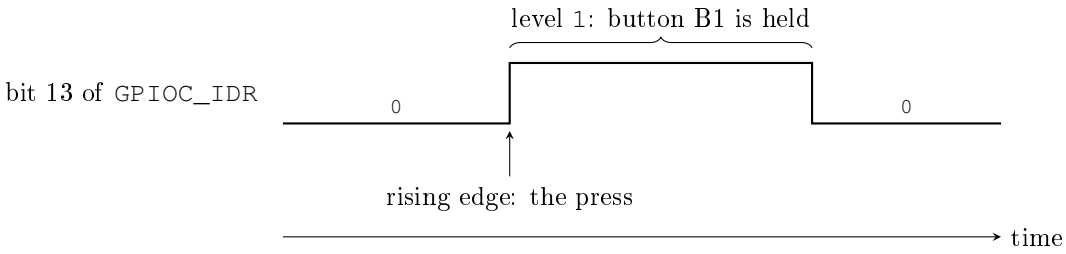


Figure 7: The level and the rising edge of one press of button B1.

A program that uses the level does its action again and again while button B1 is held. A program that uses the edge does its action one time for each press. To find an edge, compare each value with the previous value, which you keep in a variable. This method assumes that one press gives exactly one rising edge. A mechanical button does not always do this.

Contact bounce

When a mechanical switch closes, the elasticity of the contact material makes the contacts separate and connect again many times, for a short period. Then a stable connection occurs. Thus, the voltage at the pin changes between ground and V_{DD} several times before it becomes stable at its final level (Figure 8). A similar transient occurs when the switch opens. This is *contact bounce*, and it occurs in all mechanical switches. Its duration is different for different switches, and for different operations of the same switch.

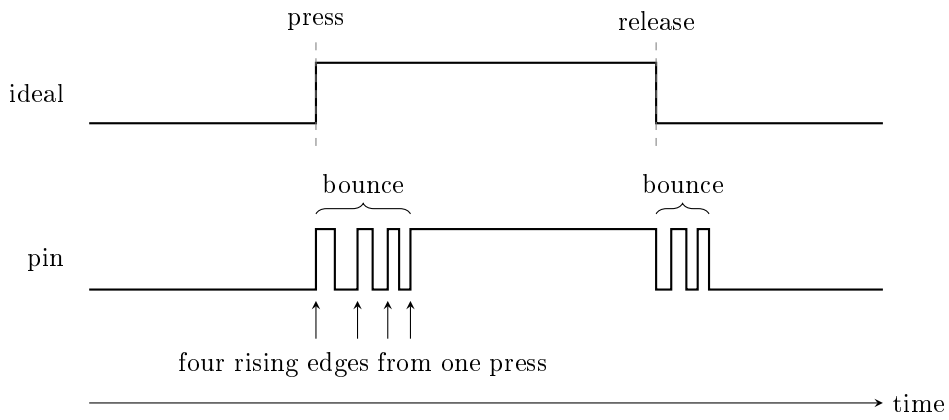


Figure 8: Contact bounce (not to scale). The ideal signal changes level one time at each press and one time at each release. The real signal at the pin changes between the two levels several times before it is stable.

The Schmitt trigger corrects a slow or noisy signal near a threshold. It does not remove contact bounce: each bounce is a full transition between ground and V_{DD}, and it crosses the two thresholds. Thus, the input data register records each bounce as a valid change of level. A program that reads the pin at a rate that is high enough finds each bounce. As a result, an edge detector counts one press as several presses.
There are two methods to remove contact bounce:

- **Hardware filter.** A resistor (resistance R) and a capacitor (capacitance C) between the contact and the pin make a low-pass filter. The voltage of the pin then changes slowly: the *time constant* $\tau = RC$ gives the time of a change. A bounce that is much shorter than τ ends before the voltage crosses the input threshold. Thus, it does not reach the input data register.
- **Software debounce.** The program accepts a new input level only after the level stays the same for longer than the bounce.

Pre-Laboratory Exercises for Part "B"

3. **Read the inputs and drive LD2.** Part "B" reads button B1 (PC13) and a second input (PA10, with its internal pull-down). It drives LD2 through `GPIOA_BSRR`. Find in RM0440:
   (a) the bit of `RCC_AHB2ENR` that enables the clock of port C (RM0440 section 7.4.15, p. 306).
   (b) the bits of `GPIOC_MODER` that hold the mode of PC13, and the two-bit value that makes PC13 an input (RM0440 section 9.4.1, p. 355).
   (c) the hexadecimal value that you write to `GPIOA_BSRR` to turn LD2 on, and the value that turns LD2 off (RM0440 section 9.4.7, p. 357).
   (d) why the two writes do not change the other pins of port A.
   (e) the bits of `GPIOA_PUPDR` that hold the setting of PA10, and the two-bit value that selects a pull-down (RM0440 section 9.4.4, p. 356).
   (f) the reset value of `GPIOA_PUPDR`, and the pins that change if a plain store writes only your pull-down value to the register.

Task B1: Control the blink speed with the user button

Write the program that Part "B" describes. Use a rising edge to find each press.

```
/* USER CODE BEGIN PD */
#define RCC_AHB2ENR    (*(volatile uint32_t *)0x4002104CUL)
#define GPIOA_MODER    (*(volatile uint32_t *)0x48000000UL)
#define GPIOA_BSRR     (*(volatile uint32_t *)0x48000018UL)
#define GPIOC_MODER    (*(volatile uint32_t *)0x48000800UL)
#define GPIOC_IDR      (*(volatile uint32_t *)0x48000810UL)

#define SLOW_HALF_PERIOD   3000000UL   /* passes, slowest speed */
#define FAST_HALF_PERIOD    187500UL   /* passes, fastest speed */
/* USER CODE END PD */

/* USER CODE BEGIN PV */
uint32_t half_period = 0;    /* passes per half-period; 0 = not started */
uint32_t countdown   = 0;    /* passes left in this half-period */
uint32_t led_on      = 0;    /* 1 while LD2 is lit */
uint32_t previous    = 0;    /* B1's level on the previous pass */
/* USER CODE END PV */
```

Listing 3: Task B1: Register macros, speed limits, and variables. Put the define lines in USER CODE BEGIN PD, and the variables in USER CODE BEGIN PV.

Steps.

1. Copy the code above into the two sections that its comments give.
2. In `USER CODE BEGIN 2`, which runs one time, enable the GPIOA and GPIOC clocks. Configure PA5 as an output and PC13 as an input. Clear each field before you set it. Turn LD2 off through `GPIOA_BSRR`.
3. In the `while (1)` loop, write one pass of the loop. The pass:
   - reads button B1 from `GPIOC_IDR`.
   - on a rising edge, starts the blink or changes to the next speed.
   - stores this value in `previous`.
   - after the blink starts, decrements `countdown`. When `countdown` is zero, the pass toggles LD2 through `GPIOA_BSRR` and loads `countdown` from `half_period` again.

Task B2: Use D2 as the button

Two jumper wires replace the button. Put one wire into D2 (PA10) and the other wire into a 3V3 socket. When the free ends of the wires make contact, the circuit closes: this is a press. When the ends separate, this is a release. **Do not connect a wire to a 5V socket.** The code below configures PA10 as the input. The other code stays as in Task B1.

```
/* USER CODE BEGIN PD: add these two macros */
#define GPIOA_PUPDR   (*(volatile uint32_t *)0x4800000CUL)
#define GPIOA_IDR     (*(volatile uint32_t *)0x48000010UL)

  /* USER CODE BEGIN 2: add after the Task B1 setup */
  GPIOA_MODER &= ~(0x3UL << (10 * 2));      /* MODE10 = 00: input */
  GPIOA_PUPDR &= ~(0x3UL << (10 * 2));      /* PUPD10: clear */
  GPIOA_PUPDR |=  (0x2UL << (10 * 2));      /* PUPD10 = 10: pull-down */

    /* while (1): replace the line that reads B1 */
    uint32_t now = (GPIOA_IDR >> 10) & 1UL;   /* D2: 1 = in contact */
```

Listing 4: Task B2: The three changes that configure PA10 as the input. Each comment gives the location of the change in main.c.

Steps.

1. Make the three changes. Build the project. Flash the board.
2. Put one wire into D2 and the other wire into 3V3. Hold one wire in each hand, and bring the two free metal ends into contact several times. LD2 starts to blink, and each contact changes the blink rate.
3. Probe PA10. Capture the signal when the wires make contact and when they separate. Measure the settling time of each.

Task B3: Debounce D2

Make sure that the program counts each contact exactly one time. To do this, debounce D2. Accept a new level of PA10 only after it stays the same for longer than the bounce from Task B2.

```
/* USER CODE BEGIN PD */
#define DEBOUNCE_PASSES   /* your value */
/* USER CODE END PD */

/* USER CODE BEGIN PV */
uint32_t accepted = 0;       /* debounced level of PA10 */
uint32_t disagree = 0;       /* passes PA10 has differed from it */
/* USER CODE END PV */
```

Listing 5: Task B3: The debounce threshold, which you calculate in step 3, and its two variables.

Steps.

1. Make contact between the wires ten times. Record the number of contacts that change the speed by more than one step.
2. Find the duration of one pass of your loop. At the slowest speed, measure the half-period of LD2 on PA5 with the oscilloscope. Divide it by `SLOW_HALF_PERIOD`.
3. Calculate `DEBOUNCE_PASSES` from this duration and the longer of the two settling times from Task B2. Include a safety margin. Record the calculation.
4. Replace the edge detection in your loop. Count each pass on which PA10 is different from `accepted`. Set the count to zero on each pass on which the two are the same. When the count is equal to `DEBOUNCE_PASSES`, copy the level of PA10 into `accepted`. Change the speed only if `accepted` is then `1`.
5. Build the project. Flash the board. Make contact between the wires ten times again. Include short contacts and contacts that slide. Each contact must change the speed by exactly one step.

Part "C": The HAL

Background for Part "C"

In Part "C", the functions of the HAL do the register accesses of Parts "A" and "B". Your program builds the Morse code lock with these functions. In Morse code, the durations of the presses and of the pauses between them encode each letter. Thus, a measurement of time identifies each letter.

The HAL

The hardware abstraction layer (HAL) is the library of C functions from ST for the peripherals of the STM32 (UM2570). Table 4 gives the functions that this laboratory uses. The HAL has two advantages:

- **Abstraction.** The program gives a port and a pin. The function finds the registers and the bit positions, and accesses them.
- **Portability.** ST supplies a HAL for each STM32 family. Each HAL has the same function names, with the register addresses of its family. Thus, you can move a program that uses the HAL for the G4 to a different STM32 with few changes. For register-level code, you must examine each address again, as Fact 1 shows for the F4.

Function	What it does	Registers
`__HAL_RCC_GPIOx_CLK_ENABLE()`	Enables the clock of port x	`RCC_AHB2ENR`
`HAL_GPIO_Init()`	Configures pins: mode, pull, output type, speed	`MODER`, `PUPDR`, ...
`HAL_GPIO_ReadPin()`	Returns the level of a pin	`IDR`
`HAL_GPIO_WritePin()`	Sets or clears a pin	`BSRR`, `BRR`
`HAL_GetTick()`	Returns the time since reset, in milliseconds	none

Table 4: The HAL functions that this laboratory uses. `HAL_GPIO_WritePin()` sets a pin through `BSRR` and clears it through `BRR`, the bit reset register of the port (RM0440 section 9.4.11, p. 361).

Time measurement with the HAL

The time base of the HAL is the SysTick timer of the Cortex-M4. `HAL_Init()` starts this timer. The timer then increments a counter each millisecond, independently of your program. `HAL_GetTick()` returns the value of the counter, a `uint32_t`. To measure an interval, record the counter when the event starts. At a later time, subtract the recorded value from the counter:

```
uint32_t start = HAL_GetTick();              /* the event starts */
uint32_t elapsed = HAL_GetTick() - start;    /* later: time in ms */
```

The two variables must be `uint32_t`, as the counter. The subtraction is then unsigned, and it gives the correct interval also after the counter goes back to 0 (Exercise 5).

Morse code

Morse code shows each letter as a sequence of two signals: a short *dot* and a long *dash* (Table 5). All durations are in units of one dot:

- The dash is three units long.
- The pause between the signals of one letter is one unit long.
- The pause between letters is three units long.

A	`.-`	**H**	`....`	**O**	`---`	**V**	`...-`
B	`-...`	**I**	`..`	**P**	`.--.`	**W**	`.--`
C	`-.-.`	**J**	`.---`	**Q**	`--.-`	**X**	`-..-`
D	`-..`	**K**	`-.-`	**R**	`.-.`	**Y**	`-.--`
E	`.`	**L**	`.-..`	**S**	`...`	**Z**	`--..`
F	`..-.`	**M**	`--`	**T**	`-`		
G	`--.`	**N**	`-.`	**U**	`..-`		

Table 5: The International Morse code for the letters A to Z. The dot is `.` and the dash is -.

Pre-Laboratory Exercises for Part "C"

4. **Encode your name.** Use the Morse code table (Table 5). Write your first name in Morse code as the string that your program compares in Task C1. Write the dots and dashes of each letter, with one space between letters. Do not put a space at the start or at the end. For example, YUVAL is `"-.-- ..- ...- .- .-.."`.

5. **Find the limit of the counter.** `HAL_GetTick()` returns a `uint32_t` counter that increments each millisecond.
   (a) Calculate the time, in days, after which the counter goes back to 0.
   (b) An event starts when the counter is `0xFFFF FFF0`. At a later time, `HAL_GetTick()` returns `0x0000 0010`. Calculate `HAL_GetTick() - start` as a `uint32_t`. Compare it with the number of milliseconds between the two times.

Task C1: Enter your name in Morse code

Your program operates as follows:

- LD2 is on while button B1 is held.
- If a press is shorter than `DOT_MAX_MS`, it is a dot. If it is longer, it is a dash.
- If a pause is longer than `LETTER_GAP_MS`, it ends a letter. If it is longer than `NAME_GAP_MS`, it ends the name.
- The program then compares the entered name with your name. If they are the same, LD2 blinks quickly for `VERDICT_MS`. If they are different, LD2 stays on for `VERDICT_MS`.

```
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PD */
#define DOT_MAX_MS      300U   /* shorter press: dot; longer: dash */
#define LETTER_GAP_MS   800U   /* a longer pause ends a letter */
#define NAME_GAP_MS    2500U   /* a longer pause ends the name */
#define VERDICT_MS     2000U   /* how long LD2 shows the verdict */
/* USER CODE END PD */

/* USER CODE BEGIN PV */
static const char name_morse[] = ". ...- .-";  /* EVA: use your name */
static char keyed[64];           /* dots, dashes, letter spaces */
/* USER CODE END PV */
```

Listing 6: Task C1: The include directive, the time limits, your name, and the buffer for the input. Put each part in the section that its comment gives.

Steps.

1. In `USER CODE BEGIN MX_GPIO_Init_2`, enable the clocks of ports A and C with `__HAL_RCC_GPIOA_CLK_ENABLE()` and `__HAL_RCC_GPIOC_CLK_ENABLE()`. Then use `HAL_GPIO_Init()` to configure PA5 as a push-pull output and PC13 as an input. Do not use an internal pull resistor on PC13. Button B1 has an external pull-down resistor on the board.
2. Copy the code above into the sections that its comments give. Replace the example in `name_morse` with the Morse code of your name from Pre-Laboratory Exercise 4.
3. In the `while (1)` loop, write one pass of the loop. Use `HAL_GetTick()` as the only time reference. The pass:
   - reads button B1 with `HAL_GPIO_ReadPin()`, and turns LD2 on while button B1 is held.
   - records the time of each press. At each release, the pass adds a dot or a dash to `keyed`, as the duration of the press sets.
   - during a pause, adds a space to `keyed` after `LETTER_GAP_MS`. After `NAME_GAP_MS`, it removes the last space from `keyed`. Then it compares `keyed` with `name_morse`, shows the result for `VERDICT_MS`, and clears `keyed`.
4. Build the project. Flash the board. Enter your name. Then enter it with one incorrect letter. Make sure that the board rejects it.
5. Enter one letter. Pause the program in the debugger. Compare the contents of `keyed` with that letter in Table 5.

Demonstration and Sign-off

Keep these items, and be ready to show them to the instructor at the test of this lab:

- Your answers to Pre-Laboratory Exercises 1 to 5.
- For each task, the items of Table 6. An image of the oscilloscope screen can be a photo or a screenshot.

Task	Copy of `main.c`	Oscilloscope image
A1	`lab1partAtask1.c`	PA5: one half-period of LD2.
A2	`lab1partAtask2.c`	PA5: one half-period, with the count in SRAM.
B1	`lab1partBtask1.c`	No image.
B2	`lab1partBtask2.c`	PA10: two images, the bounce when the wires make contact (a press), and the bounce when they separate (a release).
B3	`lab1partBtask3.c`	PA5: one half-period at the slowest speed.
C1	`lab1partCtask1.c`	No image.

Table 6: The items to keep for each task.